

PRACTICAL WEEK8

TASK 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
GNU nano 8.7.1 memory_alloc.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr1 = (int *)malloc(5 * sizeof(int));
    if (arr1 == NULL) {
        printf("malloc failed\n");
        return 1;
    }
    printf("Allocated arr1 using malloc at address: %p\n", (void *)arr1);

    int *arr2 = (int *)calloc(5, sizeof(int));
    if (arr2 == NULL) {
        printf("calloc failed\n");
        return 1;
    }
    printf("Allocated arr2 using calloc at address: %p\n", (void *)arr2);

    arr1 = (int *)realloc(arr1, 10 * sizeof(int));
    if (arr1 == NULL) {
        printf("realloc failed\n");
        return 1;
    }
    printf("Reallocated arr1 using realloc at new address: %p\n", (void *)arr1);

    free(arr1);
    printf("Freed arr1 successfully.\n");

    return 0;
}
```

```
rishill10@ASUS16X: ~/Week8_ x + v
rishill10@ASUS16X:~/Week8_Practical$ nano memory_alloc.c
rishill10@ASUS16X:~/Week8_Practical$ gcc -g memory_alloc.c -o memory_alloc
./memory_alloc
valgrind --leak-check=full ./memory_alloc
Allocated arr1 using malloc at address: 0x648e61ab4010
Allocated arr2 using calloc at address: 0x648e61ab4440
Reallocated arr1 using realloc at new address: 0x648e61ab4460
Freed arr1 successfully.
==2161== Memcheck, a memory error detector
==2161== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2161== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2161== Command: ./memory_alloc
==2161==
Allocated arr1 using malloc at address: 0x4a91040
Allocated arr2 using calloc at address: 0x4a914e0
Reallocated arr1 using realloc at new address: 0x4a91540
Freed arr1 successfully.
==2161==
==2161== HEAP SUMMARY:
==2161==   in use at exit: 20 bytes in 1 blocks
==2161== total heap usage: 4 allocs, 3 frees, 1,104 bytes allocated
==2161==
==2161== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2161==   at 0x4857E43: calloc (vg_replace_malloc.c:1678)
==2161==   by 0x400124C: main (memory_alloc.c:12)
==2161==
==2161== LEAK SUMMARY:
==2161==   definitely lost: 20 bytes in 1 blocks
==2161==   indirectly lost: 0 bytes in 0 blocks
==2161==   possibly lost: 0 bytes in 0 blocks
==2161==   still reachable: 0 bytes in 0 blocks
==2161==   suppressed: 0 bytes in 0 blocks
==2161==
==2161== For lists of detected and suppressed errors, rerun with: -s
==2161== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
rishill10@ASUS16X:~/Week8_Practical$
```

TASK 2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
rishil10@ASUS16X: ~/Week8_ × + ▾
GNU nano 8.7.1 copy_on_write.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int *data = (int *)malloc(sizeof(int));
    *data = 0;

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated and initialized.\n");
    printf("Address of data: %p\n", (void *)data);

    printf("\nCreating child process using fork()...\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Address of data in child: %p\n", (void *)data);
        printf("Before modification: data[0] = %d\n", data[0]);

        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);
        printf("Child modified the memory.\n");
        printf("Copy-on-write creates a private copy of the modified page.\n");

        sleep(5);
        free(data);
        exit(0);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent data[0] = %d\n", data[0]);

        sleep(2);

        printf("After child modification, parent data[0] = %d\n", data[0]);
        printf("Parent still has its original data.\n");

        wait(NULL);
        free(data);
    }

    return 0;
}
```

```
rishill10@ASUS16X: ~/Week8_ × + ▾
rishill10@ASUS16X:~/Week8_Practical$ nano copy_on_write.c
rishill10@ASUS16X:~/Week8_Practical$ gcc copy_on_write.c -o copy_on_write
./copy_on_write
Parent PID: 2279
Memory allocated and initialized.
Address of data: 0x63da7b868010

Creating child process using fork()...

Parent Process
Parent PID: 2279
Child PID: 2280
Parent data[0] = 0

Child Process
Child PID: 2280
Address of data in child: 0x63da7b868010
Before modification: data[0] = 0
After modification: data[0] = 9999
Child modified the memory.
Copy-on-write creates a private copy of the modified page.
After child modification, parent data[0] = 0
Parent still has its original data.
rishill10@ASUS16X:~/Week8_Practical$ |
```